Statistical Methods in AI – Monsoon 2026

Assignment 1

Deadline: 30 August 2026, 11:59 P.M.

Instructors: Prof Anoop M. Namboodiri, Prof Saikiran Bulusu

General Instructions

- Your assignment must be implemented in Python.
- Clearly label and organize your code, including comments that explain the purpose of each section and key steps in your implementation. Ensure that your files are well-structured, with headings, subheadings, and explanations as necessary.
- Make sure to test your code thoroughly before submission to avoid any runtime errors or unexpected behavior.
- Your assignment will be evaluated not only based on correctness but also on the quality of code, the clarity of explanations, and the extent to which you've understood and applied the concepts covered in the course.
- We are aware of the possibility of submitting the assignment late in GitHub Classroom using various hacks. Note that we will have measures in place accordingly and anyone caught attempting to do the same will be given a straight zero in the assignment.

AI Tools Usage Instructions (Mandatory if Applicable)

We are aware how easy it is to write code and solve questions with the help of LLM services, but we strongly encourage you to figure out the answers yourself. If you use any AI tools such as ChatGPT, Gemini, Claude, etc., to assist in solving any part of this assignment:

- **If you are unable to explain any part of the solution/code during evaluations, that solution/code will be considered plagiarized and you will be penalized.** You must also be able to briefly explain how you modified or verified the AI-generated content.
- You must include a **shareable link** to the AI Tool chat history or a **screenshot** of the relevant conversation. **Do NOT share any private or sensitive personal information** in the AI Tool conversations you include.

Submission Instructions

- Submit a self-contained **Jupyter notebook** containing all code, plots, and printed tables.
- Report all the analysis, comparison and any metrics in the notebook or a separate report that is part of the submission itself. No external links to cloud storage files, wandb logs or any other alternate will be accepted as part of your submission. Only the values and visualizations as part of your commits will be graded.
- Use your institute email ID to generate a **personalized random seed**:
  - Get the part before `@` in your IIITH email (e.g., `username` from `username@program.iiit.ac.in`).
  - Use the SHA-256 hash of this string to ensure uniqueness:

```
import hashlib

seed = int(hashlib.sha256(username.encode()).hexdigest(), 16) % (2**32)
```

  - Use this `seed` in all random number generators (e.g., `np.random.default_rng(seed)`).
- All plots must include your **email username** in the title or filename.

```
plt.text(
    0.95, 0.95, "username",
    ha='right', va='top',
    transform=plt.gca().transAxes,
    fontsize=10, color='gray', alpha=0.7
)
```

Submission Policy (GitHub Classroom Assignments)

- **Minimum Progress Requirement:** You must push at least **two meaningful commits on different days** prior to the deadline.
- **Commit Timing Check:** If over 80% of commits are made within the last 24 hours before the deadline, a **10% penalty** will be applied.
- **Commit Quality:** Commits must reflect actual progress. Non-informative or placeholder commits will not count.
- **Final Submission:** Your final grade is based on the latest commit before the deadline, but commit history will be reviewed.

Guidelines for Implementation and Code Design

1. Use object-oriented programming.
2. Use appropriate visualization libraries such as `matplotlib`, `seaborn`, or `plotly`. Each plot must include a **title, x-label, y-label, and legend (if applicable)**. Answers with missing labels or legends will receive **0 marks**.
3. Keep visualization and computation logic in **separate functions**.
4. Add `docstrings` to methods explaining what they do.
5. Use reproducible random sampling using the given `seed`.

1 Automated Essay Scoring System

Automated essay scoring is important in settings such as competitive examinations, educational assessment, and large-scale evaluation. The objective of this assignment is to develop an Automated Essay Scoring (AES) system using handcrafted statistical features and the K-Nearest Neighbors (KNN) algorithm. You are provided with the Dataset.

The dataset contains the following columns:

- **essay_id:** unique identifier for each essay
- **essay:** the essay text
- **score:** the human-assigned essay score, ranging from 1 to 6

Your task is to extract meaningful statistical features from the essay text, preprocess the resulting dataset, implement KNN for essay score prediction, and evaluate the performance of the resulting system.

1.1 Feature Engineering [15 marks]

The goal of this question is to explore features that may help distinguish essays with different scores. You have to design and implement handcrafted statistical features that capture different characteristics of an essay. You must compute the features directly from the essay text.

You are **not allowed** to use NLP libraries, pretrained models, language models, or other libraries/models specifically designed to extract these features. For example, you may consider the following features:

1. Total number of words
2. Total number of sentences
3. Number of short words

4. Number of medium words
5. Number of long words
6. Number of short sentences
7. Number of medium sentences
8. Number of long sentences

You should explore the dataset and design additional meaningful handcrafted statistical features based on the essay text. You may implement a feature extraction class as follows:

```
class EssayFeatureExtractor:

    def __init__(self):
        """Initialize the feature extractor."""
        pass

    def extract_features(self, essay: str) -> dict:
        """Extract all handcrafted features from an essay."""
        pass

    def extract_dataset_features(self, essays) -> pd.DataFrame:
        """Extract features for all essays in the dataset."""
        pass
```

The extracted features should be numerical and suitable for subsequent preprocessing and use with the KNN algorithm. Clearly explain the motivation and interpretation of each feature used.

1.2 Dataset Preprocessing [10 marks]

1.2.1 (a) Train–Validation–Test Split [3 marks]

Split the dataset into training, validation, and test sets in an 80:10:10 ratio. Report the number of samples in the training, validation, and test sets.

1.2.2 (b) Feature Normalization [3 marks]

Normalization is important for KNN because it uses distance to find the nearest data points. If features have different scales, larger-valued features can dominate the distance calculation.

Normalize the numerical features using **Z-score standardization**.

The Z-score transformation is given by:

$$z = \frac{x - \mu}{\sigma}$$

where x is the original feature value, and μ and σ are the mean and standard deviation of the feature calculated from the training set. The normalization parameters must be learned only from the training data and then applied to the validation and test data.

1.2.3 (c) Feature Visualization [4 marks]

Visualize the handcrafted features using appropriate plots such as histograms, bar charts, box plots, scatter plots, or correlation plots. At least **four meaningful visualizations** should be provided.

1.3 K-Nearest Neighbors [20 marks]

In this part, you will implement an essay scoring system using the K-Nearest Neighbors (KNN) algorithm. The input to the model will be the handcrafted feature vector of an essay, and the output will be its predicted essay score.

1.3.1 (a) Implementation [15 marks]

Implement KNN **from scratch** for predicting the essay score. You may implement a KNN class as follows:

```
class KNN:

    def __init__(self, k=5, distance="euclidean"):
        """Initialize the KNN essay scoring model."""
        ...

    def fit(self, X_train, y_train):
        """Store the training feature vectors and scores."""
        ...

    def predict(self, X):
        """Predict essay scores for the given feature vectors."""
        ...
```

1.3.2 (b) Hyperparameter Tuning [5 marks]

Experiment with different distance metrics and values of K. For example, you may experiment with:

$$K \in \{1, 3, 5, 7, 9, 11, 15, 21\}$$

and the following distance metrics:

- Euclidean distance
- Manhattan distance
- Cosine distance

For every combination of distance metric and K, evaluate the model on the validation set. You should report the validation performance for all configurations in a table similar to the following:

Distance Metric	K	Validation Performance
Euclidean	1	–
Euclidean	3	–
Euclidean	5	–
Euclidean	7	–
Manhattan	1	–
Manhattan	3	–
Manhattan	5	–
Manhattan	7	–

Identify the distance metric and value of K that gives the best validation performance. Briefly explain how the performance changes as the value of K changes and how the choice of distance metric affects the results.

1.4 Evaluation [5 marks]

Evaluate the final KNN model on the test set using the best configuration obtained from the validation experiments.

Report the following evaluation metrics:

1.4.1 (a) Mean Absolute Error

The Mean Absolute Error is defined as:

$$MAE = \frac{1}{n}\sum_{i=1}^{n}|y_i - \hat{y}_i|$$

where y_i is the actual essay score and $\hat{y}_i$ is the predicted essay score.

1.4.2 (b) Root Mean Squared Error

The Root Mean Squared Error is defined as:

$$RMSE = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}$$

1.4.3 (c) R^2 Score

The coefficient of determination is defined as:

$$R^2 = 1 - \frac{\sum_{i=1}^{n}(y_i - \hat{y}_i)^2}{\sum_{i=1}^{n}(y_i - \bar{y})^2}$$

where $\bar{y}$ is the mean of the actual essay scores.

1.4.4 (d) Pearson Correlation Coefficient

The Pearson correlation coefficient measures the linear correlation between the actual and predicted essay scores:

$$r = \frac{\sum_{i=1}^{n}(y_i - \bar{y})(\hat{y}_i - \bar{\hat{y}})}{\sqrt{\left(\sum_{i=1}^{n}(y_i - \bar{y})^2\right)\left(\sum_{i=1}^{n}(\hat{y}_i - \bar{\hat{y}})^2\right)}}$$

2 Probability Basics and Bayesian Learning

General constraints, applying throughout unless a question says otherwise:

- `np.random` and `random` are banned everywhere — Question 2.1.1 makes you the random number generator.
- `scipy.stats` is banned inside every implemented function. `scipy.special` utilities (`erfinv`, `gammaln`, `comb`, `betaincinv`) are allowed where the question says so — they are special functions, not pre-built shortcuts that would trivialize the question.
- `sklearn` (or any library implementing Naive Bayes, KNN, evaluation metrics, etc.) is banned throughout, with a single exception: part (d) of Question 2.3.4, where it is used only as a baseline to compare against your own implementation.

2.1 A Sampling Toolkit From Scratch [24 marks]

Instead of relying on `np.random` and `scipy.stats` for uniform generation, CDF inversion, and quantile computation, you build the whole toolkit yourself, from the raw pseudo-random bit stream up.

2.1.1 Uniform Generator (LCG) [3 marks]

Question

Implement a **linear congruential generator** (LCG) that produces pseudo-random numbers without using `np.random` or `random`, via the MINSTD recurrence

$$x_{k+1} = (a \cdot x_k) \bmod m, \qquad a = 48271, \qquad m = 2^{31} - 1.$$

Implement `lcg_uniform_generator(low, high, num_samples, seed)`:

1. Initialize the state as `seed % m` (if that is 0, set it to 1 — a zero state would get stuck at zero forever).
2. Iterate the recurrence `num_samples` times, storing `state / m` after each update.
3. Rescale the collected values from $(0, 1)$ to $[\text{low}, \text{high})$ and return them as a NumPy array.

Allowed: plain Python and NumPy array operations only.

2.1.2 Gaussian Sampler [4 marks]

Question

A Gaussian $X \sim \mathcal{N}(\mu, \sigma)$ has CDF

$$y = F(x) = \frac{1}{2}\left[1 + \operatorname{erf}\left(\frac{x-\mu}{\sigma\sqrt{2}}\right)\right].$$

(a) Algebraically invert this CDF for x in terms of y (you may use erf^{-1}, i.e. `scipy.special.erfinv`).

(b) Implement `inverse_cdf_gaussian(y, mu, sigma)` using your derived formula.

(c) Implement `gaussian_generator(mu, sigma, num_samples, seed)` by pushing `num_samples` values from your Question-2.1.1 generator on $(0, 1)$ through `inverse_cdf_gaussian`.

Allowed: NumPy, `scipy.special.erfinv`. *Not allowed:* `scipy.stats`, `np.random`.

2.1.3 Binomial Sampler [5 marks]

Question

A Binomial(n, p) variable has PMF

$$p_X(k) = \binom{n}{k} p^k (1-p)^{n-k}.$$

Being discrete, its "inverse CDF" is the quantile function

$$Q(y) = \min\{k \in \{0, \ldots, n\} : F(k) \geq y\}.$$

(a) Implement `binomial_pmf(k, n, p)`.

(b) Implement `binomial_quantile(y, n, p)` by building the CDF via a cumulative sum over the PMF and searching it (e.g. `np.cumsum` + `np.searchsorted`) — do not call `scipy.stats.binom`.

(c) Implement `binomial_generator(n, p, num_samples, seed)` composing your Question-2.1.1 generator with your quantile function.

Allowed: NumPy, `scipy.special.comb`. *Not allowed:* `scipy.stats`.

2.1.4 Image Sampler — Sampling a 2D Empirical Density [6 marks]

Question

A grayscale image is a 2D empirical density: normalizing its pixel intensities over the (row, col) grid gives a joint PMF $P(\text{row}, \text{col})$. By the chain rule, $P(\text{row}, \text{col}) = P(\text{row}) \cdot P(\text{col} \mid \text{row})$ — so you can sample it with exactly the discrete quantile-function machinery from Q2.1.3, applied twice.

(a) Implement `row_marginal(image)`, returning

$$P(\text{row}) = \frac{\sum_{\text{col}} \text{image}[\text{row}, \text{col}]}{\sum \text{image}},$$

and `col_conditional(image, row)`, returning

$$P(\text{col} \mid \text{row}) = \frac{\text{image}[\text{row}, :]}{\sum_{\text{col}} \text{image}[\text{row}, :]}.$$

(b) Implement `image_generator(image, num_samples, seed)`: for each of `num_samples` draws, use your Q2.1.1 generator together with the Q2.1.3-style cumsum/searchsorted quantile lookup to (i) sample a row from the row marginal's CDF, then (ii) sample a column from that row's conditional CDF. Return the array of (row, col) pairs.

(c) Run `image_generator` on the provided `paw_print.png` (100×100 grayscale, provided) with `num_samples = 5000`, and scatter-plot the (col, row) pairs — flipping the row axis — to show the point cloud reconstructing the image.

(d) Using `empirical_cdf` and `max_cdf_distance` from Q2.1.5, validate your sampler: compare your samples' empirical row marginal against the image's true row marginal over a set of evaluation rows.

Allowed: NumPy; matplotlib or PIL for reading the PNG and plotting. *Not allowed:* `scipy.stats`, `np.random`, or any prebuilt 2D density/histogram sampler (e.g. `scipy.stats.rv_histogram`).

2.1.5 Generator Validation [6 marks]

Question

Implement `empirical_cdf(sample, x)` — the fraction of `sample` at or below each point in `x` — and `max_cdf_distance(sample, cdf_function, eval_points)`, the worst-case gap

$$D = \max_j \left| \hat{F}_n(t_j) - F(t_j) \right|$$

between your generator's empirical CDF and a distribution's theoretical CDF over a set of evaluation points. Use both to check the Uniform, Gaussian, and Binomial generators from Q2.1.1, Q2.1.2 and Q2.1.3 against their theoretical CDFs.

2.2 Naive Bayes Classifier for Mixed Features [34 marks]

Setup (provided, not implemented): a synthetic dataset of four dog breeds, each described by four features drawn from three different distribution families — `height`, `weight` $\sim$ Gaussian; `bark_days` $\sim$ Binomial$(n = 30, p)$; `ear_head_ratio` $\sim$ Uniform — split 70/30 into train and test sets.

For a real-world alternative to the synthetic dog data, see the AKC Dog Breeds dataset on Kaggle, which gives physical measurements (height, weight) and behavioral trait scores —

including a 1–5 "Barking Level" — for over 200 breeds recognized by the American Kennel Club. Note that it is aggregated **per breed**, not per individual dog, so it can't be swapped in directly — you would need to treat each breed's scores as fixed parameters, or introduce your own per-individual variation around them, to fit this assignment's per-dog classification setup.

2.2.1 Maximum-Likelihood Estimators [7 marks]

Question

Implement MLE parameter estimators for all three families, given a 1-D sample:

- **Gaussian**: $\hat{\mu} = \bar{x}$ and
$$\hat{\sigma} = \sqrt{\frac{1}{N}\sum_i (x_i - \bar{x})^2}$$
— note the $\frac{1}{N}$: the MLE variance divides by N, not $N-1$.
- **Binomial** with n known:
$$\hat{p} = \frac{\bar{x}}{n}.$$
- **Uniform**:
$$\hat{a} = \min_i x_i, \qquad \hat{b} = \max_i x_i.$$

Each function must return plain Python floats. *Allowed:* NumPy only.

2.2.2 Model Fitting [8 marks]

Question

Implement `compute_training_params(df, features, feature_distributions)`, fitting per-class parameters and log-priors from a training dataframe. It must work generically over however many classes and features are present — do not hard-code the breed count or feature names; dispatch on the `feature_distributions` mapping and loop over `sorted(df["breed"].unique())`.

2.2.3 MAP Prediction [9 marks]

Question

Implement `log_posterior_scores` and `predict_class`. For a sample $\mathbf{x} = (x_1, \ldots, x_n)$ and classes $C_1, \ldots, C_m$, compute

$$\text{score}_i(\mathbf{x}) = \log P(C_i) + \sum_{k=1}^{n} \log P(x_k \mid C_i)$$

for every class in log-space (using your Question-2.1.6 and Question-2.2.2 functions), and return the class with the highest score.

2.2.4 Evaluation Metrics [10 marks]

Question

Implement `confusion_matrix(y_true, y_pred, labels)` and `precision_recall_f1(confusion_matrix)` (macro-averaged) from scratch — no `sklearn.metrics`. Use them to evaluate the classifier from Q2.2.1–Q2.2.3 on the held-out test split.

2.3 Spam Detection [22 marks]

Setup (provided, not implemented): a labeled (`text`, `spam`) email dataset, split into train and test sets.

2.3.1 Text Statistics [10 marks]

Question

Implement `fit_spam_counts(df)`, returning:

- `word_freq`: {`word`: {`"spam"`: `s`, `"ham"`: `h`}}, the number of spam/ham emails (in `df`) containing that word (a document count, not a raw token count).
- `class_freq`: {`"spam"`: ..., `"ham"`: ...}, the number of emails of each class.

Fit **only on the training split** — the test emails must stay untouched until evaluation.

2.3.2 Smoothed Classifier [12 marks]

Question

Implement two functions.

`stable_sigmoid(t)` — a numerically stable two-branch sigmoid that never exponentiates a positive number:

$$\sigma(t) = \begin{cases} \dfrac{1}{1+e^{-t}}, & \text{if } t \geq 0, \\ \dfrac{e^{t}}{1+e^{t}}, & \text{if } t < 0. \end{cases}$$

`naive_bayes_spam_prob(text, word_freq, class_freq, alpha=1.0)` — lowercase the text, take the **set** of its words, accumulate the two log-scores starting from the log-priors, using the Laplace-smoothed likelihood

$$P(w \mid c) = \frac{\text{count}_c(w) + \alpha}{N_c + 2\alpha}, \qquad \alpha > 0$$

for every word that exists in `word_freq` (skip out-of-vocabulary words), and return

$$\sigma\left(\text{score}_{\text{spam}} - \text{score}_{\text{ham}}\right).$$

2.4 Bayesian Parameter Estimation [12 marks]

Every parameter estimated in Section 2 so far has been a single MLE point value, with no way to express prior belief or report confidence. This section does the genuinely Bayesian thing instead: a prior distribution over a probability parameter, updated into a posterior via Bayes' theorem, rather than collapsed straight to a point estimate — as distinct from the "Naive Bayes" classifier of Section 2.3, which (despite the name) never once maintains a belief distribution over anything.

2.4.1 Bayesian Estimation of a Bark-Day Rate [12 marks]

Question

Work with the dog breed dataset from Section 2.2. Each dog's `bark_days` count k_i is a draw from Binomial$(n = 30, p)$. Placing a Beta(α_0, β_0) prior on p gives, by conjugacy, a Beta$(\alpha_{\text{post}}, \beta_{\text{post}})$ posterior with

$$\alpha_{\text{post}} = \alpha_0 + \sum_i k_i, \qquad \beta_{\text{post}} = \beta_0 + \sum_i (n - k_i).$$

Implement three functions.

(a) `beta_binomial_posterior(alpha_prior, beta_prior, observations, n_trials)` — apply the update above to a 1-D array of per-dog `bark_days` counts. Return `(alpha_post, beta_post)`.

(b) `posterior_point_estimates(alpha_post, beta_post)` — return `{"mean": ..., "mode": ..., "variance": ...}` for the posterior:

- mean $= \dfrac{\alpha}{\alpha + \beta}$
- mode (MAP) $= \dfrac{\alpha - 1}{\alpha + \beta - 2}$ when $\alpha, \beta > 1$; at the boundary (0 or 1) if only one exceeds 1; `float("nan")` if neither does.
- variance $= \dfrac{\alpha\beta}{(\alpha + \beta)^2(\alpha + \beta + 1)}$

(c) `credible_interval(alpha_post, beta_post, level=0.95)` — the central credible interval, using `scipy.special.betaincinv(alpha, beta, q)` (the Beta quantile function) for the two tail quantiles. `scipy.stats` is banned here too — `betaincinv` is a special function, not a "fit a Beta and read off the answer" shortcut.

Report the posterior mean, mode, variance and 95% credible interval for the bark-day rate p, and compare the posterior mean against the raw MLE proportion $\bar{k}/n$ computed directly from the training split. Comment on what the prior is doing when the training split is large, and what it would do if you had only a handful of dogs.